How to Build a Chess Agent

This guide describes six approaches to building a chess agent, progressing from a random-move baseline to a neural network trained on human games.

The shared foundation

Every chess agent performs the same fundamental task: given a position, it returns a single legal move. Regardless of the approach chosen, an agent consists of three components.

- **Board representation and rules:** a component that determines which moves are legal, calculates all possible legal moves, applies and reverses moves, and detects checkmate, stalemate and draws. The Python library python-chess (`pip install chess`) provides this functionality; alternatively, a team may implement its own move generator.
- **Decision logic:** the code that selects a move. This is the component in which the six approaches differ.
- **UCI (Universal Chess Interface):** a standard text-based protocol through which the agent communicates with chess interfaces such as Cute Chess, which are used to make agents play a game of chess with each other. The interface sends the current position and remaining time, and the agent responds with its chosen move (for example, `bestmove e2e4`).

It should be noted that the rules component only identifies which moves are legal; it does not assess their quality. All judgement regarding move selection is the responsibility of the agent's decision logic.

Way 1: Random and greedy agents

The simplest agents consider at most one move ahead. They are weak and are the most basic forms of chess agents.

A **random agent** selects any legal move at random.

A **greedy agent** evaluates every legal move by the position it produces, typically counting material only, and plays the move with the highest score. It captures undefended pieces but is easily trapped, as it does not consider the opponent's reply.

Counting material only means that the agent judges a position solely by summing the values of each side's pieces (pawn 1, knight 3, bishop 3, rook 5, queen 9) and subtracting one total from the other. Piece placement, king safety and threats are ignored entirely.

The consequence can be illustrated with an example. Suppose the agent's queen is attacked by an enemy pawn:

- **Moving the queen to safety:** both sides retain the same pieces, so the score is 0.
- **Making any other quiet move:** the queen is still on the board immediately afterwards, so the score is also 0.

Since both options score equally, the agent has no reason to save its queen and may leave it to be captured. Similarly, if the queen can capture a pawn that is defended, the agent scores the capture as +1 and plays it, only to lose the queen (−9) on the following move. This weakness arises because the agent never considers the opponent's reply, and it is addressed by the minimax search described in Way 2.

How to build it

1. Load the current position using python-chess.
2. For a random agent, select a move from the list of legal moves:

```
import random
move = random.choice(list(board.legal_moves))
```

3. For a greedy agent, define a material evaluation using standard piece values (pawn 1, knight 3, bishop 3, rook 5, queen 9):

```
def material(board):
    values = {chess.PAWN: 1, chess.KNIGHT: 3, chess.BISHOP: 3,
              chess.ROOK: 5, chess.QUEEN: 9}
    score = 0
    for piece_type, v in values.items():
        score += v * len(board.pieces(piece_type, chess.WHITE))
        score -= v * len(board.pieces(piece_type, chess.BLACK))
    return score
```

4. Apply each legal move with `board.push(move)`, record the evaluation, reverse it with `board.pop()`, and return the move with the best score for the side to move.

Way 2: Minimax search

Minimax enables an agent to look several moves ahead under the assumption that both sides play optimally. This single technique transforms a greedy agent into a basic engine capable of recognising simple tactics.

The algorithm constructs a game tree comprising every move available to the agent from the list of legal moves provided by python chess library, every reply available to the opponent, and so on to a fixed depth. Positions at the bottom of the tree are scored by an evaluation function. These scores are then propagated upward: the agent selects the maximum score on its own turns and assumes the opponent selects the minimum on theirs.

The principal limitation is the size of the tree. A typical position offers approximately 35 legal moves, so a search to depth 4 examines roughly 1.5 million positions.

How to build it

1. Reuse the evaluation function from Way 1, returning scores from White's perspective.
2. Implement a recursive search function:

```
def minimax(board, depth, maximizing):
    if depth == 0 or board.is_game_over():
        return evaluate(board)
    best = -10**9 if maximizing else 10**9
    for move in board.legal_moves:
        board.push(move)
        score = minimax(board, depth - 1, not maximizing)
        board.pop()
        best = max(best, score) if maximizing else min(best, score)
    return best
```

The function operates as follows:

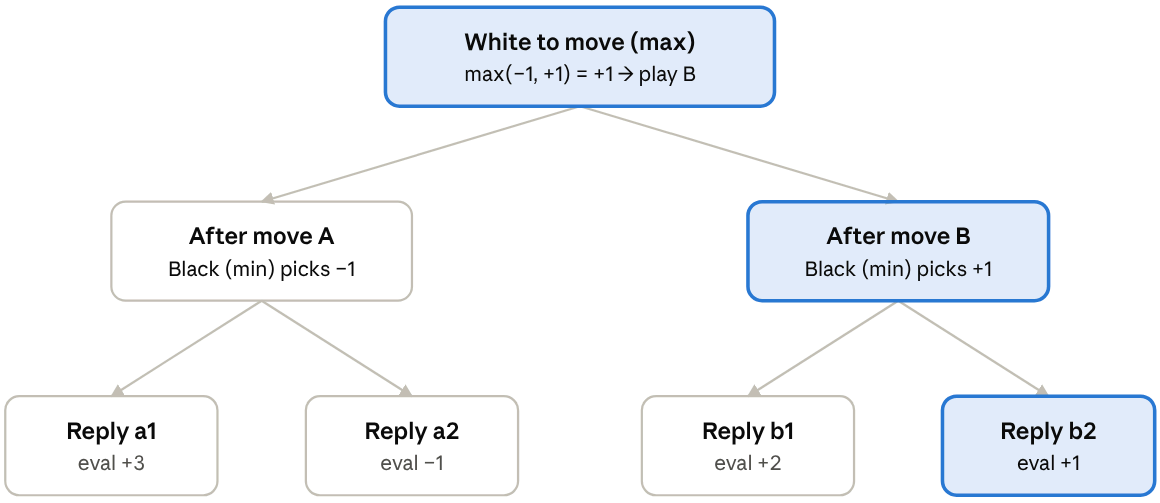
- **Parameters:** `depth` is the number of moves remaining to look ahead. `maximizing` is `True` when it is White's turn (White seeks the highest score) and `False` when it is Black's turn (Black seeks the lowest score).
- **Stopping condition:** when the depth reaches zero or the game has ended, the function stops searching and returns the evaluation of the current position.

- **Initial best score:** `best` is set to an extremely unfavourable value for the side to move (very low for White, very high for Black), so that the score of any real move will replace it.
- **Exploring each move:** for every legal move, the function plays the move with `board.push()`, calls itself with one less level of depth and the other side to move, and then reverses the move with `board.pop()`.
- **Selecting the best:** White retains the highest score encountered and Black the lowest.
- **Result:** the function returns the best score achievable from the current position, assuming both sides play optimally.

Because the function calls itself with alternating sides, it explores every sequence of moves down to the chosen depth. Scores from the final positions are then passed back up the tree, with each side selecting the outcome most favourable to it. For example, at depth 2 the agent considers each of its own moves, then each possible reply, and chooses the move whose worst-case reply leaves it in the best position.

The recursion tree below illustrates a search to depth 2, in which White and Black each have two candidate moves.

White plays move B because its worst case is better



Scores flow upward: Black takes its lowest reply, White takes the highest of Black's results.

minimax recursion tree · depth 2, two moves per side

Although move A leads to the single highest score (+3), Black would respond with reply a2, leaving White at −1. Move B guarantees at least +1 whatever Black plays, so minimax selects move B.

3. At the root, call the search for each legal move and play the move with the best resulting score.
4. Assign large scores to checkmate positions and a score of zero to draws, so that the agent pursues mates and avoids being mated.

Way 3: Alpha-beta pruning and search enhancements

Alpha-beta pruning produces the same result as minimax while omitting branches that cannot affect the final decision. This allows a substantially deeper search within the same time. Classical chess engines are built upon alpha-beta search combined with a set of well-established enhancements.

How alpha-beta pruning works

The central idea is that once a move is proven to be worse than an option already found, there is no need to discover exactly how bad it is.

Consider the recursion tree from Way 2, but suppose the agent examines move B first:

1. **Move B is searched fully.** Black's best replies leave White at +1, so White now knows it can guarantee a score of at least +1. This guaranteed value is called **alpha**.
2. **Move A is searched next.** Black's first reply examined happens to be a2, which scores −1.
3. **The search stops early.** Black can already hold White to −1 after move A, and Black's other replies can only lower that score further, never raise it. Since −1 is worse than the +1 already secured with move B, White will never choose move A. The remaining replies to move A are therefore skipped, or *pruned*.

Beta is the same idea from Black's perspective: the lowest score Black is already guaranteed. Whenever a branch is proven worse for the side to move than its alpha or beta, that branch is abandoned.

An everyday analogy: when choosing between two restaurants, if you already know restaurant B serves a good meal, and you learn that restaurant A's best available dish is worse, you need not read the rest of restaurant A's menu.

Pruning never changes the move selected; it only avoids unnecessary work. Its effectiveness depends heavily on move ordering: when strong moves are examined first, as in this example, far more branches can be skipped. With good ordering, alpha-beta can search roughly twice as deep as minimax in the same time.

How to build it

1. **Alpha-beta pruning:** extend the minimax function with two bounds, alpha and beta, representing the best scores each side is already guaranteed. Abandon a branch as soon as it is proven to be worse than an alternative.
2. **Move ordering:** examine the most promising moves first, such as captures, checks and the best move from a previous search. Pruning is considerably more effective when strong moves are searched early.
3. **Iterative deepening:** search to depth 1, then depth 2, and so on until the allotted time expires. This ensures a move is always available and improves move ordering at each successive depth.
4. **Quiescence search:** at the end of the main search, continue examining captures until the position is stable. This prevents the agent from misjudging positions in the middle of an exchange.
5. **Transposition table:** store the results of positions already evaluated, since the same position frequently arises through different move orders.
6. **Time management:** allocate thinking time for each move based on the remaining clock time supplied by the interface.

Way 4: Hand-crafted evaluation

Ways 1 to 3 rely on a basic evaluation that counts material only, treating every knight as worth 3 regardless of circumstances. In practice, the value of a piece depends heavily on context: a knight in the centre of the board is far more useful than one trapped in a corner, and a king that should hide during the middlegame must become active in the endgame. This approach improves the evaluation function by replacing fixed piece values with values that reflect the piece's position, the phase of the game, and other strategic factors such as pawn structure, mobility and king safety. The search remains unchanged; the agent simply becomes better at judging the positions it reaches.

The search determines how far ahead the agent looks, whereas the evaluation function determines how accurately it judges the resulting positions. A classical evaluation is a weighted formula of chess principles, with each weight chosen by the developer.

```
score = (100 * pawns + 320 * knights + 330 * bishops
         + 500 * rooks + 900 * queens
         + 10 * mobility + 30 * passed_pawns
         - 25 * king_exposure)
```

Because the evaluation is executed millions of times during a search, it must be computationally efficient. A simple, fast evaluation combined with a deep search generally outperforms a sophisticated but slow one.

How to build it

Add the following components to the evaluation, broadly in this order:

1. **Material:** assign piece values in centipawns (a pawn equals 100).
2. **Piece-square tables:** apply a bonus or penalty for each piece on each square, for example rewarding centralised knights or a castled king.
3. **Mobility:** reward pieces according to the number of moves available to them.
4. **Pawn structure:** penalise doubled and isolated pawns, and reward passed pawns.
5. **King safety:** account for the pawn shield in front of the king and the presence of enemy pieces nearby.
6. **Game phase:** maintain separate middlegame and endgame values and blend them according to the material remaining, as the king should be sheltered early and active late.

Initial weights may be taken from established values documented on the Chess Programming Wiki.

Way 5: Tuning a classical engine

Tuning is the first machine-learning approach in this guide. The evaluation formula from Way 4 is kept exactly as it is, but its weights are no longer chosen by hand: they become parameters learned from data, in much the same way that a straight line can be fitted to a set of points.

Why hand-chosen weights fall short

In Way 4, the developer decides, for example, that a passed pawn is worth 30 centipawns and a bishop pair 40. These figures are educated guesses. Whether a passed pawn is truly worth 30, 45 or 60 is difficult to judge, and an engine may contain hundreds of such weights that interact with one another. Tuning replaces these guesses with values that demonstrably predict the outcome of real games more accurately.

Method 1: Texel tuning (learning weights from game results)

Texel tuning determines the weights by asking how well the evaluation predicts who actually won.

1. **Collect labelled positions.** Gather a large number of positions, typically several million, from completed games. Each position is labelled with the final result of its game: 1 for a White win, 0.5 for a draw and 0 for a Black win. Only *quiet* positions should be used, meaning positions with no captures or checks pending, since the evaluation cannot judge an exchange in progress.
2. **Convert scores into predicted results.** An evaluation score in centipawns is turned into a predicted probability of White winning using a sigmoid function, where K is a scaling constant fitted to the data before tuning begins:

```
P(\text{White wins}) = \frac{1}{1 + 10^{-K \cdot \text{score} / 400}}
```

A score of 0 therefore predicts 0.5 (an even game), while large positive scores approach 1.

3. **Measure the error.** For every position, compare the predicted probability with the actual result and square the difference. The average of these squared differences across all positions is the error to be minimised.
4. **Adjust the weights.** Change each weight slightly up and down and keep whichever change reduces the error. Repeating this over all weights, many times, gradually moves every weight towards the value that best predicts the results. Gradient descent can be used instead to adjust all weights simultaneously and more quickly.
5. **Stop when the error no longer improves.** The resulting weights are the tuned evaluation.

As an illustration, suppose that positions in which one side holds the bishop pair are won more often than the current evaluation predicts. The error for those positions is consistently too low, so tuning raises the bishop-pair bonus until the predictions match the observed results.

Method 2: Self-play testing (verifying improvements)

A lower prediction error does not guarantee stronger play, so every tuned version must be verified in actual games. The new version plays a large number of fast games against the previous version, and a statistical procedure known as the Sequential Probability Ratio Test (SPRT) determines when enough games have been played to conclude whether the change is a genuine improvement. Only changes that pass this test are kept.

How to build it

1. Expose every evaluation weight as an adjustable parameter rather than a fixed constant.
2. Collect quiet training positions with known game results, either from the Lichess open database or from the engine's own games.
3. Fit the scaling constant K, then implement Texel tuning to minimise the prediction error.
4. Validate each tuned version through self-play matches against the previous version using SPRT, with a tool such as cutechess-cli or fastchess.

Way 6: Supervised neural network

Rather than encoding chess knowledge by hand, this approach trains a neural network to learn it from millions of human games. It is the most practical machine-learning approach for a competition of this kind.

The network does not know the rules of chess; python-chess continues to supply the legal moves. The network only judges moves and positions, through two outputs:

- **Policy:** a probability for every possible move (every combination of starting and destination square), indicating how likely a strong player is to choose it. Because this includes moves that are illegal in the current position, the agent considers only the probabilities of the legal moves supplied by python-chess.
- **Value:** a single number predicting the result of the game, from +1 (White wins) through 0 (draw) to −1 (Black wins). This predicted result serves as the measure of how good a position is.

The trained network can be used in one of two ways:

- **Policy only:** the agent plays the legal move with the highest policy probability. This resembles a player moving on instinct without calculation, and it is prone to tactical errors.
- **Network as the evaluation function (recommended):** the alpha-beta search from Way 3 examines every legal move for both sides down to the depth limit, and the network's value output replaces the hand-written evaluation when scoring the final positions. In this arrangement the network does not choose moves; it judges positions, while the search provides the look-ahead. A capture that appears attractive but loses to a reply two moves later is therefore correctly rejected.

How to build it

1. **Obtain data:** download games from the Lichess open database (database.lichess.org), filtering by player rating to select the level of play to be learned.
2. **Encode positions:** represent each board numerically, typically as 12 planes of 8×8 values (one per piece type and colour), together with the side to move and castling rights.
3. **Prepare labels:** record the move played as the policy target and the final game result as the value target.
4. **Train the network:** build a small convolutional network in PyTorch or TensorFlow and train it on several million positions, using a GPU or Google Colab.
5. **Integrate the network:** for the recommended approach, rewrite the evaluation function so that it encodes the position, passes it through the network and returns the value output, leaving the search unchanged. Ensure the score's perspective matches the search, keep the network small since each call is slower than a hand-written evaluation, and optionally use the policy output to order moves so that more branches are pruned.